

Ứng dụng tư tưởng loại suy trong giải bài

Zhang Li

1999-12-05

Nguồn gốc: The application of analogical thinking in problem solving

IOI China candidate team papers / 2000 / Zhang Li.pdf

Từ khóa. Tư tưởng; loại suy; tính tương tự; tương ứng.

Tóm tắt

Loại suy là một cách nghĩ cố gắng thiết lập liên hệ giữa bài toán chưa biết và bài toán đã biết, từ đó dùng phương pháp giải đã biết để xử lý bài toán mới. Bài viết trước hết phân tích các ví dụ cụ thể để chỉ ra rằng giải bài bằng loại suy không chỉ là nhận thấy sự giống nhau bề mặt, mà quan trọng hơn là thiết lập quan hệ tương ứng giữa bài toán đã biết và bài toán chưa biết. Sau đó, thông qua một ví dụ khác, bài viết bàn về quan hệ giữa sự tương tự bề mặt và sự tương ứng bên trong, đồng thời chỉ ra rằng quá trình dùng loại suy để giải bài là quá trình đi từ sự tương tự bề mặt đến quan hệ tương ứng một-một.

Dẫn nhập: bắt đầu từ chỗ quen thuộc

Khi đối diện một bài toán mới, ta nên bắt đầu phân tích và giải nó như thế nào?

Hãy bắt đầu từ chỗ quen thuộc. Đây là cách con người thường dùng trong đời sống: hy vọng bài toán khó trước mắt giống hệt một bài toán từng giải, hoặc ít nhất cũng tương tự với nó; nhờ vậy ta có thể có kinh nghiệm đáng tham khảo. Khi gặp một bài toán xa lạ, cố gắng liên hệ nó với một bài toán quen thuộc, rồi mượn tri thức và phương pháp quen thuộc để giải bài mới, là một ý nghĩ tự nhiên.

Tư tưởng tìm liên hệ giữa bài toán chưa biết và bài toán đã biết này có thể gọi là **loại suy**. Nói chính xác hơn, “nếu giữa các bộ phận tương ứng của hai hệ thống tồn tại một quan hệ nhất quán nào đó, thì hai hệ thống ấy có thể loại suy với nhau” [1].

Ta nên hiểu “quan hệ nhất quán” trong định nghĩa trên như thế nào?

Nếu chỉ hiểu đơn giản rằng “tồn tại một quan hệ nhất quán nào đó” là một dạng tương tự mơ hồ, thì loại suy hoàn toàn bị quy về cảm giác chủ quan của con người. Cảm giác chủ quan kiểu “hình như đã gặp” ấy không thể làm căn cứ để phân tích và giải bài. Tuy nhiên, tư tưởng loại suy thật sự được dùng rộng rãi trong nhiều loại vấn đề, điều này cho thấy bản chất của loại suy là một “quan hệ nhất quán” đáng tin cậy và thuyết phục hơn sự giống nhau trên bề mặt. Trên thực tế, loại suy được xây dựng trên một ánh xạ tương ứng một-một giữa hai bài toán. Phần đầu của bài viết này nhằm trình bày chính quan hệ “nhất quán” ở tầng bản chất ấy.

Dù vậy, liên hệ bản chất giữa hai bài toán không dễ thu được. Trong phân tích ban đầu, điều người ta thường nhận ra vẫn là sự giống nhau bề mặt, thậm chí chỉ giống nhau về câu chữ. Hy vọng trực tiếp có

được quan hệ tương ứng và chuyển hóa lẫn nhau giữa hai bài toán là không thực tế. Vì thế, sự tương tự bề mặt quan sát được lúc đầu tuy có phần không chắc chắn, nhưng ít nhất nó có thể chỉ ra một hướng phân tích, để từ đó thử thiết lập quan hệ tương ứng giữa các bài toán. Như phần thứ hai của bài viết sẽ trình bày, dùng loại suy để giải bài là một quá trình phân tích đi từ mơ hồ đến rõ ràng, từ bề mặt đến bản chất.

Hai phần tiếp theo sẽ bàn về quan hệ giữa sự tương tự bề mặt và sự tương ứng bản chất trong quá trình loại suy.

1 Bản chất của loại suy: quan hệ tương ứng một-một

Là một phương pháp tư duy để phân tích vấn đề, loại suy có mục đích chuyển tri thức chưa quen thành tri thức quen thuộc, chuyển bài toán chưa biết thành bài toán đã biết.

Việc chuyển hóa này được thực hiện như thế nào phụ thuộc vào cách ta loại suy giữa hai bài toán. Nếu chỉ xét sự tương tự bề mặt giữa chúng, ta rất có thể máy móc bắt chước phương pháp giải của bài toán đã biết để giải bài toán mới. Cách nghĩ này thiếu chỗ dựa nghiêm ngặt và đáng tin cậy, nên khó bảo đảm thành công trong giải bài thực tế. Ngay cả khi thành công, ta cũng chỉ biết “làm được” mà không biết vì sao làm được; do chưa phát hiện mối liên hệ bản chất giữa các bài toán, ta thường không thu được phương pháp tốt nhất. Ngược lại, nếu trong quá trình loại suy tồn tại một quan hệ tương ứng giữa hai bài toán, sao cho mọi đối tượng mô tả và thao tác trong bài toán chưa biết đều có nội dung tương ứng một-một trong bài toán đã biết, thì toàn bộ bài toán chưa biết có thể được chuyển thành bài toán đã biết thông qua ánh xạ tương ứng ấy. Khi đó ta có thể dùng phương pháp giải bài toán đã biết để xử lý nó.

Ví dụ sau đây thuộc loại đó.

“Chia tách số nguyên” và “phân tích nhân tử”

Chia tách số nguyên. Cho một số nguyên dương n . Hãy chia n thành tổng của một nhóm các số nguyên dương nhỏ hơn n , không xét thứ tự sắp xếp của nhóm số đó. Hỏi có tất cả bao nhiêu cách chia? [2]

Đây là một bài toán đếm tổ hợp quen thuộc. Có hai cách giải.

Mô hình liệt kê đệ quy

Ta mô tả bài toán như sau: đếm tất cả các bộ số nguyên dương $(a_1, a_2, a_3, \dots, a_m)$ thỏa

$$n = a_1 + a_2 + a_3 + \dots + a_m \quad (1 \leq a_1 \leq a_2 \leq \dots \leq a_m < n).$$

Vì thế có thể dùng đệ quy để lần lượt xác định từng a_i , từ đó tìm mọi nghiệm và thống kê tổng số.

Gọi $D(k, n)$ là số cách chia n thành tổng của một nhóm số nguyên dương không nhỏ hơn k . Chẳng hạn a_1 có thể chọn một số nguyên bất kỳ trong khoảng $1, \dots, [n/2]$; phần còn lại $n - a_1$ được chia tiếp thành tổng của các số nguyên không nhỏ hơn a_1 . Do đó

$$D(1, n) = \sum_{1 \leq a_1 \leq [n/2]} D(a_1, n - a_1).$$

Tương tự, với a_2 ,

$$D(a_1, n - a_1) = \sum_{a_1 \leq a_2 \leq [(n - a_1)/2]} D(a_2, n - a_1 - a_2) + 1,$$

trong đó tổng ứng với trường hợp chia thành nhiều hơn hai số, còn +1 ứng với trường hợp chỉ chia thành hai số. Từ đó dễ thu được công thức đệ quy

$$D(k, n) = \sum_{k \leq i \leq [n/2]} D(i, n - i) + 1.$$

Đáp án của bài toán là $D(1, n) - 1$, trừ đi cách chia tầm thường $n = n$.

Vì bài toán gốc chỉ yêu cầu tổng số cách, còn thuật toán này trong quá trình tính lại sinh ra mọi cách chia cụ thể, nên khi n lớn, hiệu quả của thuật toán liệt kê rất thấp. Trong phụ lục, chương trình tương ứng được đặt tên là DIVIDE1.PAS.

Mô hình hàm sinh

Giả sử trong nhóm số được chia, số 1 xuất hiện x_1 lần, số 2 xuất hiện x_2 lần, v.v. Khi đó bài toán có thể mô tả thành: đếm số nghiệm nguyên của phương trình bất định

$$1x_1 + 2x_2 + \dots + kx_k + \dots + nx_n = n \quad (x_i \geq 0).$$

Do đó có thể xây dựng hàm sinh để đếm số nghiệm nguyên của phương trình bất định [3].

Không chọn số 1 được biểu diễn bởi $x^0 = 1$; chọn một số 1 được biểu diễn bởi x ; chọn hai số 1 được biểu diễn bởi x^2 ; chọn k số 1 được biểu diễn bởi x^k . Theo nguyên lý cộng, mọi cách chọn số 1 được biểu diễn bởi

$$1 + x + x^2 + x^3 + \dots + x^n.$$

Không chọn số 2 được biểu diễn bởi 1; chọn một số 2 được biểu diễn bởi x^2 ; chọn hai số 2 được biểu diễn bởi x^4 ; chọn k số 2 được biểu diễn bởi x^{2k} . Do đó mọi cách chọn số 2 được biểu diễn bởi

$$1 + x^2 + x^4 + \dots + x^{2k} + \dots + x^{2\lfloor n/2 \rfloor}.$$

Tương tự, với số nguyên dương r , chọn k số r được biểu diễn bởi x^{kr} , nên mọi cách chọn r được biểu diễn bởi

$$1 + x^r + x^{2r} + \dots + x^{kr} + \dots + x^{\lfloor n/r \rfloor r}.$$

Theo nguyên lý nhân, việc đồng thời chọn các số $1, 2, \dots, n-1$ được biểu diễn bởi

$$\begin{aligned} g(x) = & (1 + x + x^2 + x^3 + \dots + x^n) \\ & (1 + x^2 + x^4 + \dots + x^{2k} + \dots + x^{2\lfloor n/2 \rfloor}) \\ & (1 + x^3 + x^6 + \dots + x^{3k} + \dots + x^{3\lfloor n/3 \rfloor}) \\ & \dots (1 + x^r + x^{2r} + \dots + x^{kr} + \dots + x^{\lfloor n/r \rfloor r}) \dots (1 + x^{n-1}). \end{aligned}$$

Vì tổng cuối cùng cần bằng n , hệ số của x^n trong khai triển của $g(x)$ chính là đáp án.

Với loại hàm sinh này, ta không thể trực tiếp dùng công thức tổng quát để tính hệ số khai triển, nhưng có thể nhân đa thức tuần tự để tìm hệ số cần thiết. Khi cài đặt, quá trình này tương đương một thuật toán trộn danh sách tuyến tính; đồng thời quá trình tính không cần sinh các nghiệm cụ thể của phương trình, nên nhanh hơn nhiều so với cách thứ nhất. Trong phụ lục, chương trình tương ứng được đặt tên là DIVIDE2.PAS.

Phân tích nhân tử. Cho một số nguyên dương n . Hãy phân tích n thành tích của các nhân tử, không tính 1 và chính n , không xét thứ tự các nhân tử. Hỏi có tất cả bao nhiêu cách phân tích?

Việc chia một số nguyên dương thành vài số nguyên dương là cách diễn đạt quen thuộc. Thực ra nó tương tự với bài toán chia tách số nguyên. Dĩ nhiên sự khác biệt giữa hai bài là rõ ràng: một bài tách thành tích, bài kia tách thành tổng. Nhưng cả hai đều đếm số cách chia một số nguyên thành vài số nguyên.

Sau khi nhận ra sự giống nhau bề mặt đó, ta xác định rằng bài toán “phân tích nhân tử” cũng có thể giải bằng liệt kê. Hãy bắt chước mô hình thứ nhất của bài chia tách số nguyên và mô tả bài toán là đếm tất cả các bộ số nguyên dương $(a_1, a_2, a_3, \dots, a_m)$ thỏa

$$n = a_1 a_2 a_3 \dots a_m \quad (2 \leq a_1 \leq a_2 \leq a_3 \leq \dots \leq a_m < n).$$

Có thể thấy mô tả của hai bài rất giống nhau, chỉ thay dấu cộng bằng dấu nhân. Lý do then chốt khiến ta có thể thay trực tiếp như vậy mà không đổi nội dung khác là: phép cộng và phép nhân đều thỏa tính giao hoán và tính kết hợp; chúng là hai phép toán rất tương tự.

Từ mô tả trên, bắt chước $D(k, n)$ của bài chia tách số nguyên, ta định nghĩa $F(k, n)$ là số cách phân tích n thành tích của một nhóm số nguyên dương không nhỏ hơn k . Khi đó dễ suy ra

$$F(k, n) = \sum_{\substack{k \leq i \leq \lfloor \sqrt{n} \rfloor \\ i|n}} F(i, n/i) + 1.$$

Đáp án là $F(2, n) - 1$, trừ đi trường hợp $n = n$.

Do dùng liệt kê, khi n có nhiều nhân tử, chương trình rất chậm. Trong phụ lục, chương trình tương ứng được đặt tên là P1.PAS. Mô hình giải trên giống phương pháp thứ nhất của bài chia tách số nguyên một cách đáng ngạc nhiên. Tuy nhiên, bắt chước máy móc cũng chỉ làm được đến đó. Nó không giải thích được vì sao hai bài toán lại giống nhau như vậy.

Dù sao, một điểm đã được nêu ra: hai bài giống nhau vì phép cộng và phép nhân giống nhau. Vậy giữa phép cộng và phép nhân rốt cuộc có quan hệ gì?

Để trả lời câu hỏi này, ta cần đưa vào định nghĩa đẳng cấu.

Giả sử có hai tập hợp A, B , và $\otimes, \oplus$ là hai phép toán nhị phân lần lượt được định nghĩa trên hai tập ấy. Khi đó A cùng phép toán $\otimes$ tạo thành một hệ đại số $\langle A, \otimes \rangle$, còn B cùng phép toán $\oplus$ tạo thành hệ đại số $\langle B, \oplus \rangle$.

Nếu tồn tại một ánh xạ một-một

$$f(x) : A \rightarrow B$$

sao cho với mọi $x \in A, y \in A$ đều có

$$f(x \otimes y) = f(x) \oplus f(y),$$

thì hai hệ đại số $\langle A, \otimes \rangle$ và $\langle B, \oplus \rangle$ được gọi là đẳng cấu.

Nếu hai hệ đại số đẳng cấu, thì hiển nhiên bài toán trong một hệ luôn có thể chuyển hóa thành bài toán trong hệ còn lại. Bây giờ chỉ cần dùng kiến thức phổ thông, ta biết rằng $\langle \mathbb{R}^+, \times \rangle$ và $\langle \mathbb{R}, + \rangle$ là đẳng cấu: đặt $f(x) = \ln x$ với $x > 0$, khi đó luôn có

$$f(ab) = f(a) + f(b).$$

Hàm logarit $\ln x$ chính là công cụ chuyển phép nhân thành phép cộng.

Có cơ sở này, ta có thể tự tin dùng hàm sinh để giải bài phân tích nhân tử.

Trước hết, giả sử số nguyên n có tổng cộng m nhân tử ngoài 1 và n , là $p_1, p_2, p_3, \dots, p_m$. Gọi số lần nhân tử p_i xuất hiện trong phân tích là x_i . Khi đó bài toán có thể mô tả thành: đếm số nghiệm nguyên không âm của phương trình

$$p_1^{x_1} p_2^{x_2} p_3^{x_3} \cdots p_m^{x_m} = n.$$

Tiếp theo, dùng $\ln x$ để chuyển phép nhân thành phép cộng; lấy logarit hai vế, ta nhận được phương trình tuyến tính tương ứng

$$x_1 \ln p_1 + x_2 \ln p_2 + x_3 \ln p_3 + \cdots + x_m \ln p_m = \ln n.$$

Muốn đếm số nghiệm nguyên không âm của nó, ta dùng nguyên lý cộng và nguyên lý nhân để xây dựng hàm sinh, tương tự đoạn tương ứng trong bài chia tách số nguyên:

$$\begin{aligned} g(x) = & (1 + x^{\ln p_1} + x^{2 \ln p_1} + x^{3 \ln p_1} + \cdots + x^{\lfloor \ln n / \ln p_1 \rfloor \ln p_1}) \\ & \cdot (1 + x^{\ln p_2} + x^{2 \ln p_2} + x^{3 \ln p_2} + \cdots + x^{\lfloor \ln n / \ln p_2 \rfloor \ln p_2}) \\ & \cdots (1 + x^{\ln p_k} + x^{2 \ln p_k} + x^{3 \ln p_k} + \cdots + x^{\lfloor \ln n / \ln p_k \rfloor \ln p_k}) \cdots \\ & \cdot (1 + x^{\ln p_m} + x^{2 \ln p_m} + x^{3 \ln p_m} + \cdots + x^{\lfloor \ln n / \ln p_m \rfloor \ln p_m}). \end{aligned}$$

Hệ số của hạng tử $x^{\ln n}$ trong khai triển của $g(x)$ chính là đáp án. Tương tự phương pháp thứ hai của bài chia tách số nguyên, dùng thuật toán trộn danh sách tuyến tính là có thể tìm được đáp án. Trong phụ lục, chương trình tương ứng được đặt tên là P2.PAS. So với thuật toán liệt kê đệ quy, hiệu quả cũng tăng lên đáng kể.

Nếu chỉ dựa vào sự tương tự bề mặt quan sát được mà không hiểu rằng giữa hệ đại số của phép nhân và phép cộng tồn tại quan hệ đẳng cấu, thì không dễ thu được mô hình dùng hàm sinh.

Điều gợi mở nhất trong ví dụ này là việc đưa vào khái niệm đẳng cấu. Để mô tả quan hệ giữa hai hệ đại số đẳng cấu, ta dùng một ánh xạ một-một $f(x) : A \rightarrow B$. Ánh xạ ấy tạo ra quan hệ tương ứng giữa hai phép toán khác nhau của hai hệ:

$$x \otimes y \text{ trong } A \quad \longleftrightarrow \quad f(x) \oplus f(y) \text{ trong } B.$$

Như vậy, mỗi bài toán trong tập A liên quan đến phép toán $\otimes$ đều tương ứng với một bài toán trong tập B liên quan đến phép toán $\oplus$. Ta có thể chuyển bài toán trong $\langle A, \otimes \rangle$ thành bài toán trong $\langle B, \oplus \rangle$ để giải.

Thực hiện sự chuyển hóa này chính là mục đích của việc dùng tư tưởng loại suy để giải bài, còn tìm quan hệ tương ứng một-một ấy là thực chất của loại suy. Chỉ cần phát hiện được quan hệ tương ứng một-một, ta có thể áp dụng phương pháp giải bài đã biết để giải bài toán mới.

Đẳng cấu chỉ là khái niệm trong hệ đại số, nhưng quan hệ tương ứng có thể áp dụng cho những bài toán khác. Vì thế giải bài bằng loại suy không bị giới hạn trong phạm vi hệ đại số. Ví dụ tiếp theo thuộc một phương diện khác, nhằm tiếp tục làm rõ quan hệ giữa biểu hiện bên ngoài và bản chất của loại suy.

2 Từ sự tương tự bề mặt đến sự tương ứng bên trong

Bản chất của loại suy là làm lộ ra quan hệ tương ứng một-một giữa bài toán đã biết và bài toán chưa biết. Nhưng như đã nói trong dẫn nhập, bản chất thường khó phát hiện, còn sự tương tự bề mặt lại được quan sát trước tiên. Vì thế không nên xem nhẹ sự tương tự bề mặt; ngược lại, nên bắt đầu phân tích theo hướng mà sự tương tự ấy chỉ ra, rồi thử phát hiện liên hệ bản chất giữa các bài toán.

Trong quá trình phân tích ví dụ sau, ta sẽ nhấn mạnh điểm này.

Bài toán mặt phẳng chia không gian

Một mặt phẳng chia không gian thành hai phần độc lập; hai mặt phẳng chia không gian thành 4 phần. Hỏi n mặt phẳng nhiều nhất có thể chia toàn bộ không gian thành bao nhiêu phần? [1]

Trường hợp trong không gian là xa lạ và không dễ mô tả. Tuy cùng là bài toán “chia”, trong tình huống bình thường ta sẽ không liên hệ nó với bài chia tách số nguyên, vì đối tượng xử lý của hai bài hoàn toàn khác nhau: số nguyên và mặt phẳng trong không gian. Vậy phải liên hệ bài toán này với tri thức đã biết bằng cách nào? Lúc này, ngay cả sự tương tự bề mặt cũng có thể cung cấp một manh mối giải bài.

Mục đích của loại suy là biến cái chưa biết, phức tạp thành cái đã biết, đơn giản. Bài toán trong không gian phức tạp, còn bài toán trên mặt phẳng thì đơn giản hơn. Hơn nữa, quan hệ giữa không gian và mặt phẳng tương tự với quan hệ giữa mặt phẳng và đường thẳng. Một mặt phẳng chia không gian thành hai phần; một đường thẳng cũng chia mặt phẳng thành hai phần.

Vì vậy ta nhận được một bài toán khác tương tự với ví dụ: n đường thẳng nhiều nhất có thể chia toàn bộ mặt phẳng thành bao nhiêu phần? Bài toán này dễ giải hơn nhiều. Cách giải dùng quy nạp toán học.

Một đường thẳng chia mặt phẳng thành hai phần, gọi là A và B . Thêm một đường thẳng mới; nó cắt đường thẳng thứ nhất và bị chia thành hai tia. Mỗi tia lại chia miền chứa nó thành hai phần, chẳng hạn A thành A_1 và A_2 . Vì vậy số miền tăng thêm 2, tổng cộng có $2 + 2 = 4$ phần. Thêm một đường thẳng

nữa; nó cắt hai đường thẳng trước đó và bị chia thành ba đoạn. Mỗi đoạn lại chia miền chứa nó thành hai phần, chẳng hạn B_1 thành B_{11} và B_{12} . Tổng cộng tăng thêm 3 phần, nên có $4 + 3 = 7$ phần.

Quá trình này có thể hình dung như sau: mỗi khi thêm một đường thẳng, các giao điểm với những đường cũ chia đường thẳng mới thành nhiều đoạn, và mỗi đoạn làm tăng thêm một miền của mặt phẳng.

Suy rộng ra, giả sử $k - 1$ đường thẳng đầu đã chia mặt phẳng thành A_{k-1} phần. Thêm đường thẳng thứ k , nó cắt $k - 1$ đường thẳng trước và bị chia thành k đoạn; mỗi đoạn chia miền chứa nó thành hai phần, nên tổng cộng tăng thêm k phần. Tổng số phần là $A_{k-1} + k$.

Do đó ta có truy hồi

$$A_1 = 2, \quad A_k = A_{k-1} + k.$$

Từ đây dễ tính được công thức tổng quát

$$A_k = 1 + \frac{(1+k)k}{2}.$$

Bài toán đường thẳng chia mặt phẳng đã được giải.

Việc giải bài toán này giúp gì cho bài “mặt phẳng chia không gian”? Vì quan hệ giữa không gian và mặt phẳng tương tự với quan hệ giữa mặt phẳng và đường thẳng, liệu có thể trực tiếp thay “mặt phẳng” bằng “không gian”, thay “đường thẳng” bằng “mặt phẳng”, rồi dùng nguyên quá trình trên để chứng minh bài toán chưa biết hay không? Hiển nhiên là không. Đó chỉ là bất chước máy móc dựa trên sự tương tự chủ quan, và là sai.

Nhưng nếu phân tích kỹ nguyên nhân sai, ta sẽ thấy điểm then chốt: hai đường thẳng cắt nhau cho một điểm, còn hai mặt phẳng cắt nhau cho một đường thẳng; k điểm chia một đường thẳng thành $k + 1$ đoạn, nhưng k đường thẳng không đơn giản chia một mặt phẳng thành $k + 1$ phần. Trên thực tế, k đường thẳng nhiều nhất chia mặt phẳng thành A_k phần.

Vì vậy điểm giống nhau thật sự giữa hai bài là: để tính một đường thẳng làm tăng số phần của mặt phẳng bao nhiêu, ta trước hết tính đường thẳng ấy bị các điểm giao với đường cũ chia thành bao nhiêu phần, tức chuyển bài toán hai chiều thành bài toán một chiều; còn để tính một mặt phẳng làm tăng số phần của không gian bao nhiêu, ta trước hết tính mặt phẳng ấy bị các đường giao với mặt phẳng cũ chia thành bao nhiêu phần, tức chuyển bài toán ba chiều thành bài toán hai chiều. Bài toán hai chiều vừa được giải, nên ngược lại bài toán ba chiều cũng được giải.

Ta vẫn dùng quy nạp toán học. Một mặt phẳng chia không gian thành hai phần. Giả sử $k - 1$ mặt phẳng đầu đã chia không gian thành B_{k-1} phần. Khi thêm mặt phẳng thứ k , nó cắt $k - 1$ mặt phẳng trước đó, tạo ra $k - 1$ đường giao. $k - 1$ đường thẳng ấy chia mặt phẳng mới thành bao nhiêu mảnh? Đây chính là bài toán vừa giải: $k - 1$ đường thẳng chia mặt phẳng thành

$$A_{k-1} = 1 + \frac{k(k-1)}{2}$$

phần. Mỗi mảnh mặt phẳng ấy lại chia miền không gian ban đầu chứa nó thành hai phần, nên tổng cộng tăng thêm

$$1 + \frac{k(k-1)}{2}$$

phần. Do đó k mặt phẳng chia không gian thành

$$B_{k-1} + 1 + \frac{k(k-1)}{2}$$

phần.

Ta nhận được truy hồi

$$B_1 = 2, \quad B_k = B_{k-1} + 1 + \frac{k(k-1)}{2}.$$

Dùng truy hồi này để viết chương trình, ta dễ tính được B_n ; ngay cả khi n rất lớn, chương trình vẫn chạy nhanh. Dĩ nhiên cũng có thể tính công thức tổng quát của B_n ; thực chất đây là tổng của một cấp số sai phân bậc hai:

$$B_n = 1 + n + \frac{n(n-1)}{2} + \frac{n(n-1)(n-2)}{6}.$$

Trong ví dụ này, quan hệ tương ứng một-một tuy không dễ biểu diễn bằng công thức như khái niệm đẳng cấu, nhưng vẫn rất rõ ràng: với một mặt phẳng bất kỳ trong n mặt phẳng, các giao tuyến do $n-1$ mặt phẳng còn lại tạo ra chia mặt phẳng ấy thành A_{n-1} phần; điều này tương ứng chính xác với lượng tăng khi mặt phẳng thứ n chia không gian. Nói cách khác,

$$\Delta B_n = B_n - B_{n-1} = A_{n-1}.$$

Trong ví dụ này, ban đầu ta không tìm được ngay bài toán quen thuộc tương tự. Nhưng từ góc độ đơn giản hóa và dễ phân tích, ta xây dựng một bài toán tương tự trên mặt phẳng. Khi ấy, sự tương tự có thể nói là loại suy giữa “khối” và “mặt phẳng”, một dạng tương tự rất mơ hồ. Hầu như không có lý do chính xác nào cho thấy loại suy như vậy có thể làm các bài toán chuyển hóa lẫn nhau; nó chủ yếu dựa trên kinh nghiệm giải bài trước đây, chẳng hạn chuyển bài toán không gian thành bài toán mặt phẳng là thủ pháp thường gặp trong hình học không gian ở bậc phổ thông. Một lý do khác là nhu cầu đơn giản hóa bài toán.

Sau khi giải xong bài toán phẳng, khó khăn tiếp theo là: làm thế nào áp dụng kinh nghiệm giải bài toán phẳng vào bài toán không gian? Nếu chỉ dựa trên sự tương tự bề mặt, việc bắt chước máy móc sẽ dẫn đến một phương pháp sai. Tuy nhiên, thử nghiệm ấy không phải vô giá trị. Khi so sánh sự khác nhau giữa lập luận đúng và lập luận sai, loại suy ban đầu vốn đơn giản và mơ hồ được hoàn thiện và làm rõ: ta loại suy “quá trình giảm từ ba chiều xuống hai chiều” với “quá trình giảm từ hai chiều xuống một chiều”. Từ đó, liên hệ giữa hai bài toán trở nên rõ ràng, và quá trình tính toán đầy đủ được suy ra.

Trên thực tế, cách giải ví dụ này rất giống phương pháp tư duy của nhiều bài đếm ba chiều trong những năm gần đây: trước hết xét trường hợp chuyển từ hai chiều về một chiều; sau đó dùng cùng phương pháp, tức loại suy, để chuyển bài toán ba chiều về bài toán hai chiều. Chiến lược này được gọi một cách hình ảnh là chiến lược “giảm chiều” [4].

Hai ví dụ “phân tích nhân tử” và “mặt phẳng chia không gian” ở trên đều được giải nhờ loại suy. Hơn nữa, ban đầu cả hai đều lấy việc tìm một sự tương tự nào đó giữa các bài toán làm manh mối phân tích; mục đích của việc này là liên hệ bài toán chưa biết với tri thức quen thuộc, từ đó cung cấp khả năng giải bài. Nhưng như đã thấy trong hai ví dụ, sự tương tự rút ra từ bề mặt bài toán thường không phải bản chất và có thể có khiếm khuyết. Vì vậy, loại suy ở tầng hiện tượng bề mặt chỉ chỉ ra một hướng khả thi cho việc giải bài. Để cuối cùng chuyển được bài toán chưa biết thành bài toán đã biết, cần làm lộ ra sự loại suy sâu hơn giữa hai bài, tức thiết lập quan hệ tương ứng từ bài toán này sang bài toán kia.

Trong toàn bộ quá trình dùng loại suy để giải bài, sự tương tự bề mặt và sự tương ứng bên trong liên hệ chặt chẽ với nhau. Tương tự nằm ở tầng nông, quan hệ tương ứng nằm ở tầng sâu. Quan hệ tương ứng là chìa khóa giải bài, còn con đường phát hiện quan hệ tương ứng bắt đầu từ sự tương tự bề mặt.

3 Tổng kết

Thi tin học liên quan đến rất nhiều thuật toán khác nhau. Với đặc điểm và kỹ thuật ứng dụng của từng thuật toán, thầy cô và các thế hệ đi trước đã có nhiều phân tích, nghiên cứu, đồng thời tổng kết được nhiều lý thuyết hoàn chỉnh. Vậy trong phương diện tư duy phân tích và giải bài, có tồn tại quy luật và phương pháp nhất định hay không? Xuất phát từ suy nghĩ ấy, bài viết này không phân tích một thuật toán cụ thể, mà mong muốn bàn về phương pháp tư duy khi phân tích vấn đề. Ở phương diện này, nhiều kỹ thuật toán học như dựng hình tư duy, quy về bài đã biết, và quy nạp đều đáng tham khảo; tư tưởng loại suy mà bài viết chủ yếu phân tích cũng là một trong số đó.

Phần thứ nhất của bài viết, thông qua ví dụ giải bài bằng loại suy, trước hết chỉ ra rằng chỉ sự tương tự bề mặt không thể thật sự phản ánh liên hệ thực chất giữa hai bài toán. Sau đó, từ định nghĩa đẳng cấu của hệ đại số, bài viết rút ra bản chất của loại suy: quan hệ tương ứng một-một giữa các bộ phận của hai hệ thống. Quan hệ tương ứng ấy làm cho một thao tác hay phép toán trong hệ này tương ứng với một thao tác hay phép toán trong hệ kia. Nhờ đó, một bài toán mới trong một hệ thống xa lạ có thể được chuyển thành bài toán quen thuộc để giải.

Quan hệ tương ứng một-một là mặt bản chất của loại suy, còn sự tương tự chủ quan là biểu hiện bên ngoài của loại suy. Đồng thời, như phần thứ hai đã trình bày, hai mặt này không hoàn toàn đối lập mà liên hệ với nhau. Không có sự dẫn dắt của tương tự chủ quan, bản chất của loại suy rất khó được phát hiện. Toàn bộ quá trình loại suy đi từ sự tương tự mơ hồ mang tính chủ quan đến quan hệ tương ứng rõ ràng ở tầng khái niệm, dần liên hệ bài toán đã biết với bài toán chưa biết, cuối cùng làm cho hai bài có thể chuyển hóa lẫn nhau, từ đó áp dụng mô hình giải của bài toán đã biết vào bài toán chưa biết.

Dùng loại suy để giải bài không phải một thuật toán cố định, mà là một phương pháp tư duy khi phân tích bài toán; vì thế nó không có khuôn mẫu bất biến, nhưng vẫn có quy luật để theo. Mục đích của loại suy là biến bài toán phức tạp thành bài toán đơn giản, biến bài toán xa lạ thành bài toán quen thuộc. Để thực hiện chuyển hóa này, ban đầu ta có thể dựa theo một sự tương tự nào đó để liên hệ hai bài toán khác nhau, rồi thử bất cứ cách giải bài đã biết để xử lý bài mới. Trong quá trình bất cứ cách ấy, nếu làm rõ được quan hệ tương ứng giữa hai bài toán, ta có thể thực hiện sự chuyển hóa giữa chúng; nếu không, nếu chỉ bất cứ cách máy móc, thường rất khó giải được bài.

Loại suy là một cách nghĩ liên hệ đủ loại thông tin và tri thức đã biết với bài toán xa lạ. Nó vừa phụ thuộc vào khả năng tư duy phân tán của thí sinh khi giải bài, vừa phụ thuộc vào độ sâu và độ rộng của tri thức mà thí sinh tiếp xúc trong học tập và luyện tập hằng ngày. Nền tảng của giải bài bằng loại suy cũng đòi hỏi thí sinh có tri thức tương tự với bài toán chưa biết; điều này cần cả căn bản vững chắc lẫn phạm vi kiến thức rộng.

Vì thế, tư tưởng giải bài bằng loại suy đặt ra cho thí sinh yêu cầu: “năng lực” cộng với “căn bản”.

References

- [1] G. Polya. *Mathematics and Plausible Reasoning*. Bản dịch tiếng Trung của Li Xincan, Wang Rishuang và Li Zhiyao, Science Press, 1984.
- [2] Jiang Xin'er. *Thiết kế thuật toán*. Tài liệu bồi dưỡng hướng dẫn viên hoạt động ngoại khóa tin học thanh thiếu niên toàn quốc, 1996.
- [3] Zhuang Xingu. *Tổ hợp học và ứng dụng trong khoa học máy tính*. Xidian University Press, 1989.
- [4] Shi Runting. *Ẩn hóa, đa chiều hóa, mở hóa*. Luận văn CTSC 1999.

A Phụ lục

So sánh hiệu quả các thuật toán cho bài chia tách số nguyên

N	Đáp án	Liệt kê đệ quy (giây)	Hàm sinh, nhân đa thức (giây)
5	7	0.00	0.00
10	42	0.00	0.00
50	204226	0.05	0.00
80	15796476	7.14	0.00
90	56634173	25	0.00
95	104651419	47	0.00
100	190569292	120	0.05

So sánh hiệu quả các thuật toán cho bài phân tích nhân tử

N	Đáp án	Liệt kê đệ quy (giây)	Hàm sinh, nhân đa thức (giây)
12	3	0.00	0.00
24	6	0.00	0.00
1048576	626	0.05	0.05
9261000	27035	0.22	0.49
1073741824	5603	0.33	0.11
1803601800	558189	10.05	3.19
479001600	2787809	16.65	2.25
518918400	2756006	18.19	3.68
1556755200	6348742	45.12	4.84

Danh sách chương trình

DIVIDE1.PAS. Thuật toán liệt kê đệ quy cho bài chia tách số nguyên.

```
PROGRAM Divide_1;
VAR
  n : INTEGER;
  total : LONGINT;      { tong so nghiem }

PROCEDURE init;        { khai tao, doc n }
BEGIN
  readln(n);
  total := 0;
END;

PROCEDURE D(k,l:INTEGER); { thu tuc de quy }
VAR i:INTEGER;
BEGIN
  FOR i := k TO l div 2 DO
    D(i,l-i);          { chia l-i thanh tong cac so khong nho hon i }
    Inc(total);         { neu l khong chia tiep, thu duoc mot nghiem }
  END;
```

```

BEGIN
    init;
    D(1,n);          { chia n thanh tong cac so khong nho hon 1 }
    writeln(total-1); { tru truong hop n=n }
END.

```

DIVIDE2.PAS. Thuật toán hàm sinh cho bài chia tách số nguyên.

```

PROGRAM Divide_2;
VAR
    n : INTEGER;
    total : LONGINT;      { tong so nghiêm }

PROCEDURE init;          { khoi tao, doc n }
BEGIN
    readln(n);
    total := 0;
END;

PROCEDURE work;          { thu tuc chinh }
TYPE
    TForm = array[0..1000] of LONGINT;
    { chi so mang la so mu }

VAR
    X1, X2 : TForm;
    i, j, k : INTEGER;
BEGIN
    fillchar(X1,sizeof(X1),0);
    FOR i := 0 TO n DO X1[i] := 1;    { X1=1+x+x2+x3+...+xn }
    FOR i := 2 TO n-1 DO
        BEGIN
            { X1*(1+xi+x2i+x3i+...+x[n/i]) duoc tam luu vao X2 roi gan ve X1 }
            X2 := X1;
            j := i;
            WHILE (j <= n) DO
                BEGIN
                    { tinh X2 = X2 + xj * X1 }
                    FOR k := 0 TO n DO X2[k+j] := X2[k+j] + X1[k];
                    j := j+i;
                END;
            X1 := X2;
        END;
    total := X1[n];      { he so cua x^n la tong so nghiêm }
END;

BEGIN
    init;
    work;

```

```
writeln(total);
END.
```

P1.PAS. Thuật toán liệt kê đệ quy cho bài phân tích nhân tử.

```
PROGRAM p1;
VAR
  n : LONGINT;
  total : LONGINT;      { tong so nghiem }

PROCEDURE init;        { khoi tao, doc n }
BEGIN
  readln(n);
END;

PROCEDURE P(k,l:LONGINT); { thu tuc de quy }
VAR i:LONGINT;
BEGIN
  FOR i := k TO Trunc(Sqrt(l)) DO
    IF l mod i = 0 THEN
      P(i,l div i);      { phan tich l/i thanh tich cac so khong nho hon i }
      inc(total);        { neu l khong phan tich tiep, thu duoc mot nghiem }
    END;
  END;

BEGIN
  init;
  P(2,n);               { phan tich n thanh tich cac so khong nho hon 2 }
  writeln(total-1);     { tru truong hop n=n }
END.
```

P2.PAS. Thuật toán hàm sinh cho bài phân tích nhân tử.

```
PROGRAM p2;
VAR
  n : LONGINT;
  total : LONGINT;      { tong so nghiem }

PROCEDURE init;        { khoi tao, doc n }
BEGIN
  readln(n);
END;

PROCEDURE work;        { thu tuc chinh }
TYPE
  TForm = array[1..1500] of RECORD
    s, exp : LONGINT;    { s la he so, exp luu co so trong so mu ln(exp) }
  END;
VAR
```

```

i : LONGINT;
F1, F2, F3 : TForm;
top1, top2, top3 : INTEGER; { do dai cua F1, F2, F3 }

PROCEDURE AddForm(p:LONGINT);
{ tinh F1 = F1 * (1+xln(p)+x2ln(p)+x3ln(p)+...)
  chi giu cac hang tu ung voi uoc cua n }
VAR
  j, k, l, m : LONGINT;
BEGIN
  j := 1;          { j = p^i }
  m := n;
  WHILE m mod p = 0 DO { khi m chua p, j*p la uoc cua n }
  BEGIN
    m := m div p;
    j := j*p;
    { tinh F1*xln(j), roi tron voi F2 va tam luu vao F3 }
    l := 1;
    top3 := 0;
    fillchar(F3,sizeof(F3),0);
    FOR k := 1 TO top1 DO
      IF m mod F1[k].exp = 0 THEN
        BEGIN
          { dua cac hang tu co so mu nho hon vao F3 truoc }
          WHILE (l <= top2) and (F2[l].exp < F1[k].exp*j) DO
            BEGIN
              inc(top3);
              F3[top3] := F2[l];
              inc(l);
            END;
          inc(top3);
          IF F2[l].exp = F1[k].exp*j THEN
            BEGIN
              F3[top3].exp := F2[l].exp;
              F3[top3].s := F3[top3].s + F1[k].s;
              inc(l);
            END
          ELSE
            BEGIN
              F3[top3].s := F1[k].s;
              F3[top3].exp := F1[k].exp*j;
            END;
          END;
        END;
      WHILE (l <= top2) DO
        BEGIN
          inc(top3);
          F3[top3] := F2[l];
          inc(l);
        END;
      END;
    END;
  END;

```

```

        END;
        F2 := F3;
        top2 := top3;      { luu ket qua tron vao F2 }
    END;
    F1 := F2;
    top1 := top2;          { luu ket qua nhan da thuc vao F1 }
END;

BEGIN
    fillchar(F1,sizeof(F1),0);
    top1 := 1;
    F1[1].s := 1;
    F1[1].exp := 1;        { F1 = 1 }
    top2 := top1;
    F2 := F1;              { F2 = F1 = 1 }
    FOR i := 2 TO trunc(sqrt(n)) DO
        IF n mod i = 0 THEN
            BEGIN
                AddForm(i);
                IF i*i <> n THEN
                    AddForm(n div i);
            END;
        i := top1;
        WHILE (i > 0) and (F1[i].exp <> n) DO dec(i);
            { he so cua x^ln(n) la tong so nghiem }
        IF i = 0 THEN total := 0 ELSE total := F1[i].s;
    END;

    BEGIN
        init;
        work;
        writeln(total);
    END.

```